

Chapter 7

System calls and user pointers

User programs cannot touch devices, other processes or the kernel's memory; the CPU refuses. Everything they need from outside their own address space, they must *ask* for — through a system call, a controlled door into the kernel. Your `pthread_create` is such a door, and every argument that comes through it is chosen by the user program: possibly wrong, possibly malicious.

7.1 User mode and kernel mode

x86 CPUs run in one of four privilege levels (rings); operating systems use two. In *kernel mode* (ring 0) every instruction is allowed. In *user mode* (ring 3) privileged instructions — `cli`, loading `cr3`, I/O ports — raise an exception, and pages whose page-table entries lack the *user* bit are inaccessible (chapter 8). The only ways from user to kernel mode are:

Interrupts from devices (timer, keyboard) — asynchronous, not caused by the running code;

Exceptions caused by the current instruction: page fault, division by zero, general protection fault;

Traps deliberate: the program executes `int n` or `syscall` to request a service.

All three enter the kernel at an address the kernel registered in advance (the *interrupt descriptor table*, or the `syscall` MSRs). The program chooses *which* service — by a number — never *where* the kernel starts executing. An interrupt gate also has a privilege level (DPL): user code may only trigger gates with DPL 3 via `int n`.

7.2 Calling conventions

	Instruction	Number / arguments	Result
Linux x86-64	<code>syscall</code>	<code>rax / rdi, rsi, rdx, r10, r8, r9</code>	<code>rax</code> , errors as <code>-errno</code>
SWEB x86-64	<code>int \$0x80</code>	<code>rax / rbx, rcx, rdx, rsi, rdi</code>	<code>rax</code>

Linux uses `r10` for the fourth argument because the `syscall` instruction itself overwrites `rcx` (return address) and `r11` (flags). SWEB passes the number plus five arguments — exactly enough for `pthread_create`'s four arguments plus the address of a start wrapper.

Errors. The Linux kernel returns `-errno` (a value between `-4095` and `-1`). The libc wrapper turns it into `-1` and stores the positive number in `errno`, a *thread-local* variable (chapter 2). Any

other value, even a “negative” address, is a valid result — hence the test `(unsigned long)r > -4096UL`.

7.3 One system call, end to end

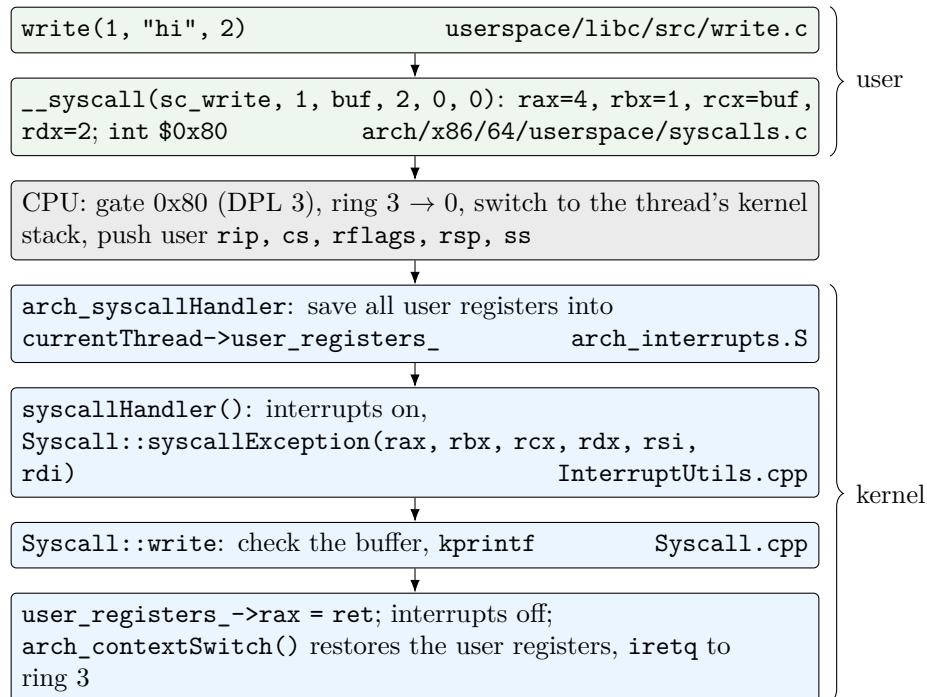


Figure 7.1: The path of `write` through SWEB (x86-64).

Note two details in figure 7.1. The kernel runs the system call on the *thread's own kernel stack*, so every thread needs one (in SWEB it is part of the `Thread` object). And interrupts are *on* during the call: a system call can be preempted, and several threads can be inside the kernel at once.

7.4 Never trust a user pointer

For `write(fd, buf, len)` the kernel reads `len` bytes at `buf`. If it did so blindly, a program could pass a kernel address and have the kernel copy kernel memory into a file — or an unmapped address and crash the kernel. Every pointer from user space must be checked:

1. **The whole range is user memory.** Not just the start: `buf + len` must also stay below the user/kernel boundary.
2. **The check cannot overflow.** `buf + len > USER_BREAK` fails when `buf + len` wraps around: `buf = 0x1000, len = 0xffffffffffff800` gives `0x800`, “below the break”. Never add two user-controlled numbers; subtract from a constant:

```
if (len > USER_BREAK || buf > USER_BREAK - len) return -EFAULT;
```

3. **The pages exist** (or the kernel handles the fault on a bad user page without dying). Linux copies with `copy_from_user`, which turns a fault into `-EFAULT`; SWEB's page-fault handler loads valid user pages on demand and ends the process otherwise.
4. **Output buffers are writable.**
5. **Strings** have no known length: copy byte by byte (page by page) into a bounded kernel buffer, checking each page as the copy reaches it — `strlen` on a user pointer *is* the unchecked read.

Not everything that looks like a pointer is one. `pthread_create`'s `arg` is never dereferenced by the kernel — it is passed to the new thread — so it must *not* be checked; `NULL` and `42` are legal. `start_routine` is not dereferenced either; it becomes an instruction pointer and at most needs to lie below `USER_BREAK`.

7.5 TOCTOU and the double fetch

Once a process has several threads, user memory can change *while* the kernel works with it. If the kernel reads a value from user memory to check it and reads it again to use it, another thread can swap it in between:

```
if (!access_ok(iov[i].base, iov[i].len)) return -EFAULT;    /* fetch 1 */
copy_to_device(iov[i].base, iov[i].len);                  /* fetch 2: may differ! */
```

The fix is simple and absolute: copy the user data into kernel memory *once*, then check and use only the copy. (Modern CPUs add SMAP, which makes the kernel fault on any user-memory access outside explicit copy routines.)

In SWEB

Syscall numbers live in `common/include/kernel/syscall-definitions.h`, shared by the kernel and the libc. Adding a system call means: a number, a `case` in `Syscall::syscallException`, a static `Syscall` method, and a libc wrapper calling `__syscall` (module 11, part 5 does it for `gettid`). SWEB's own checks are the hand-written `buffer + size > USER_BREAK` kind — including the overflow hole above. Your `pthread_create` and `pthread_join` receive user pointers (`pthread_t *`, `void **`); check them properly, and check them *before* you create or free anything.

Key points

- The user picks the service number, never the kernel entry point; gates and privilege levels enforce it.
- Linux: `syscall`, args in `rdi rsi rdx r10 r8 r9, -errno`; SWEB: `int $0x80`, number in `rax`, args in `rbx rcx rdx rsi rdi`.
- Every system call runs on the calling thread's kernel stack, with interrupts on.
- Validate the whole user range, overflow-free; copy strings with per-page checks; do not check opaque arguments.
- Fetch user data once; check and use the kernel copy.